```
class PyApplication:
  """Top-level container"""
  symbol_table: Dict[str, PyModule]

class PyModule:
  file_path: str
  qualified_name: str
  is_test: bool
  functions: List[PyFunction]
  classes: List[PyClass]
  imports: List[PyImport]

class PyImport:
  from_statement: str
  imports: List[str

class PyCallSite:
  method_name: str
  method_signature: str
  qualified_module_name: str
  receiver_type: Optional[str]
  arguments: List[PyArgument]
  argument_types: List[Optional[str]]
  argument_expr: List[str]
  return_type: str
  callee_signature: str
  is_static_call: bool
  is_constructor_call: bool
  is_library_call: bool
  is_application_call: bool
  start_line: int
  start_column: int
  end_line: int
  end_column: int

class PyFunction:
  qualified_module_name: str
  qualified_name: str
  name: str
  kind: str
  parameters: List[PyArgument]
  return_type: Optional[str]
  decorators: List[PyDecorator]
  docstring: Optional[str]
  is_test: bool
  start_line: int
  end_line: int
  call_sites: List[PyCallSite]

class PyClass:
  qualified_name: str
  class_name: str
  bases: List[str]
  decorators: List[PyDecorator]
  docstring: Optional[str]
  start_line: int
  end_line: int
  methods: List[PyFunction]
  is_test_class: bool

class PyDecorator:
  expression: str

class PyArgument:
  name: str
  annotation: Optional[str]
  type: Optional[str]
  value: Optional[Any]
  kind: str
```

Figure 6: The Tangent code analysis model.

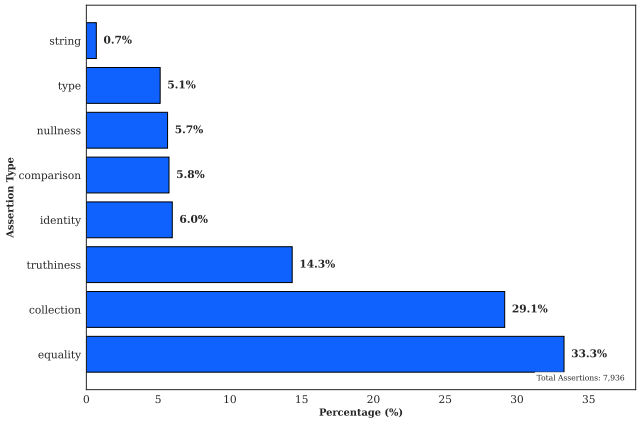

Figure 7: Distribution of assertion types.

11 Agent Test Complexity

RQ3 (Test Complexity): What is the structural complexity of agent-related tests?

In this RQ, we analyze the structural complexity of agent-related tests. We define a test complexity model and use static analysis to extract key properties of test methods to populate it [1]. At the assertion level, we record assertion types and code. We also collect standard complexity metrics, including non-comment lines of code (NCLOC) and cyclomatic complexity, together with call-related metrics such as constructor and library calls. At the method level, we extract information on mock usage, identify fixture methods (setup and teardown), and analyze helper methods, including their

```
class Assertion(BaseModel):
  assertion_type: AssertionType
  assertion_code: str

class ComplexityMetrics(BaseModel):
  ncloc: int
  cyclomatic_complexity: int

class CallMetrics(BaseModel):
  number_of_constructor_call: int
  number_of_library_call: int

class MethodInfo(BaseModel):
  qualified_class_name: str
  qualified_module_name: str
  method_signature: str
  method_body: str

class MethodData(
  MethodInfo,
  ComplexityMetrics,
  CallMetrics):
  number_of_mocking_used: int

class FixtureMethod(MethodData):
  is_setup: bool = False
  is_teardown: bool = False

class HelperMethod(MethodData):
  assertions: List[Assertion]
  number_of_assertions: int

class TestMethod(
  ComplexityMetrics,
  CallMetrics):
  test_name: str
  qualified_module_name: str
  file_path: str
  start_line: int
  end_line: int
  fixtures: List[FixtureMethod]
  helpers: List[HelperMethod]
  assertions: List[Assertion]
  number_of_fixtures_used: int
  number_of_helper_methods: int
  number_of_mocking_used: int
  number_of_assertions: int
```

assertions. For each test, we record its name, location, associated fixtures and helpers, and aggregate counts such as the number of fixtures, helpers, mocks, and assertions. Our analysis [1] shows that agent-related tests are generally structurally simple. The median test contains 14 NCLOC and has cyclomatic complexity one; including fixtures and helpers raises these medians only to 20 NCLOC and three. Assertions are also lightweight, with a median of two per test. Call statements are similarly limited, with medians of one constructor call and four library calls per test.

Research Directions. A recent study of Java tests [74] found that real-world tests often exercise multiple focal classes and methods and validate interactions across components. Such interaction-oriented tests appear less common in current agentic systems, highlighting the need for techniques that generate tests beyond isolated behaviors and systematically exercise interactions among components, environments, and non-agentic parts of the application.

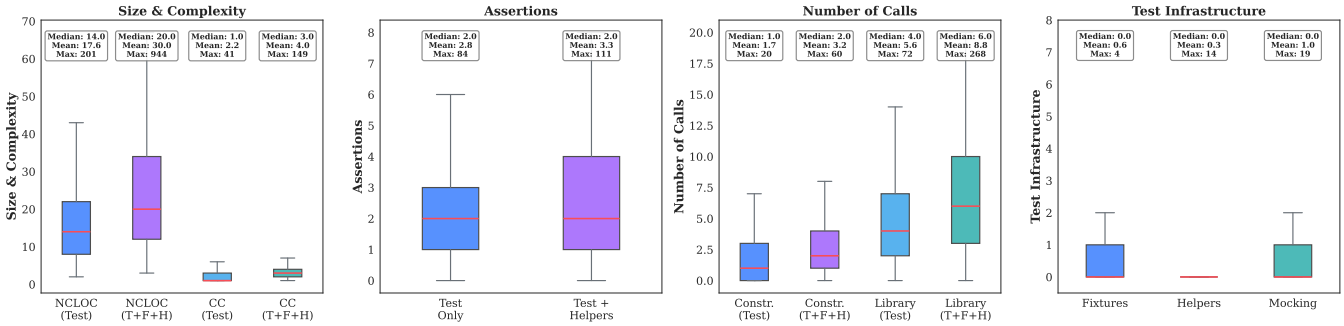

Figure 8: Complexity of agent-related tests.